

CI do SpaceLine com GitHub Actions

Escopo

O repositório tem três workflows em `.github/workflows`: `quality.yml`, `autofix.yml` e `discord-pr.yml`. O primeiro é o pipeline de qualidade usado como verificação da pull request. Os outros automatizam correções de estilo e enviam notificações; eles não substituem os checks de qualidade.

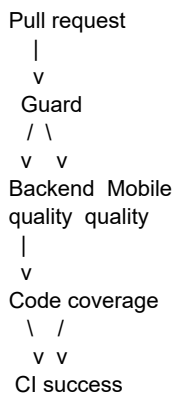
Quando o pipeline principal roda

O workflow CI dispara quando uma pull request é aberta, recebe novos commits (`synchronize`) ou é reaberta. Não há filtro por branch de destino. O grupo de concorrência usa o número da pull request (ou a referência Git como alternativa) e cancela execuções anteriores do mesmo grupo quando chega uma atualização nova.

A permissão padrão do workflow é vazia (`permissions: {}`). Cada job faz checkout do SHA da pull request. O checkout do job Guard usa `fetch-depth: 0`, necessário para a ação de validação de mensagens de commit consultar o histórico.

Visão do fluxo

Guard é a etapa inicial. Quando passa, os jobs Backend quality e Mobile quality rodam em paralelo. Code coverage depende do job de backend. No fim, CI success consolida os resultados dos quatro checks.



O job CI success usa `if: always()`: executa mesmo que uma dependência tenha falhado ou sido cancelada. Seu passo falha se algum resultado de job estiver failure ou cancelled. Assim, há um check agregado para indicar o resultado do conjunto.

Jobs e verificações

Guard

Executa em `ubuntu-latest` e prepara as ferramentas usadas pelos checks:

- valida mensagens de commit com `wagoid/commitlint-github-action@v6`;
- instala `pnpm 11.9.0` e `Node.js 22`, com cache de `pnpm`;
- configura Python 3.11, com cache de `pip` baseado em `apps/server/requirements.txt`;
- instala dependências JavaScript com `pnpm install --frozen-lockfile --ignore-scripts`;
- instala as dependências Python listadas em `apps/server/requirements.txt`;
- verifica a formatação do app mobile com `pnpm --filter mobile format:check`;
- roda `ruff check .`, `ruff format --check .` e `mypy .` no backend.

O job instala `mypy` novamente antes de executá-lo. A configuração em `apps/server/pyproject.toml` exclui testes da análise do `mypy`. O Ruff usa Python 3.11 como alvo, ignora migrations e seleciona regras das famílias E, W, F, I, B e UP, com exceções E501 e B008.

Backend quality

Depende de Guard e roda em outro runner Ubuntu. Instala as dependências Python e pytest-cov, inicia apenas o serviço spaceline-db do Docker Compose e verifica a disponibilidade do PostgreSQL. A checagem tenta até dez vezes, com intervalos de três segundos; em caso de falha, exibe os logs do container.

O teste recebe DATABASE_URL=postgresql+psycopg://postgres:postgres@localhost:5432/postgres. O workflow procura arquivos test_*.py ou *_test.py. Se encontrar, roda pytest --cov --cov-report=xml; se não encontrar, registra que não há testes e pula o pytest. Ao final, docker compose down -v é executado mesmo após falha e o arquivo apps/server/coverage.xml é enviado como artifact, com ausência permitida.

O arquivo pyproject.toml configura python_files = ["*.test.py"], enquanto o passo do workflow procura test_*.py e *_test.py. Como esses padrões não são idênticos, a descoberta do guard de testes e a descoberta do pytest podem não coincidir em alguns nomes de arquivo. Vale manter os nomes de teste conforme a configuração do pytest e conferir essa diferença quando forem adicionados testes.

Mobile quality

Depende de Guard, reinstala as dependências JavaScript e roda três verificações em sequência:

1. pnpm exec tsc --noEmit -p apps/mobile/tsconfig.json verifica tipos TypeScript; 2. pnpm --filter mobile lint executa ESLint em apps/mobile/src; 3. pnpm exec expo config --json valida a configuração do Expo, descartando a saída.

Embora o pacote mobile tenha o script test com Playwright, o workflow não o chama. Também não há etapa de build do aplicativo nem execução de E2E neste pipeline.

Code coverage

Depende de Backend quality. Baixa o artifact coverage-report para apps/server e executa codecov/codecov-action@v4 a partir desse diretório. O passo de upload do artifact no job de backend permite que o arquivo esteja ausente; por isso, quando o pytest é pulado, a disponibilidade de um relatório para o Codecov não é garantida pelo YAML.

CI success

Depende de Guard, Backend quality, Mobile quality e Code coverage. Como sempre executa e verifica falhas ou cancelamentos entre dependências, este é o resumo final do workflow.

Workflows auxiliares

Autofix

autofix.yml também é acionado em pull requests e contém três jobs independentes:

- prettier: instala pnpm 11.9.0 e Node.js 22, roda pnpm exec prettier . --write e pnpm --filter mobile exec eslint src --fix;
- ruff: configura Python 3.11, instala os requisitos do backend, roda ruff check apps/server --fix e ruff format apps/server;
- toml-sort: instala toml-sort e ordena apps/server/pyproject.toml.

Cada job entrega as mudanças pela ação autofix-ci/action@v1, com uma mensagem própria de commit. O autofix é separado do workflow de qualidade: correções automáticas não significam que todos os checks do workflow CI passaram.

Notificação no Discord

discord-pr.yml dispara quando uma pull request é aberta ou reaberta. Envia título, URL, autor e repositório para o webhook configurado em DISCORD_WEBHOOK_URL. A notificação exige o secret correspondente no repositório. Este workflow comunica a abertura da PR; não valida código nem participa das dependências de CI success.

O que o pipeline não faz

No estado atual dos arquivos de workflow, o pipeline principal não roda os testes Playwright, não faz build do aplicativo mobile, não executa pnpm test na raiz e não publica uma aplicação. O CI também não dispara diretamente em push para branches fora do contexto de pull request.

Comandos equivalentes para execução local

Na raiz do repositório:

```
pnpm --filter mobile format:check
pnpm exec tsc --noEmit -p apps/mobile/tsconfig.json
pnpm --filter mobile lint
pnpm exec expo config --json
```

No diretório apps/server:

```
ruff check .
ruff format --check .
mypy .
pytest --cov --cov-report=xml
```

Para reproduzir a etapa de banco localmente, suba o PostgreSQL e defina a URL usada pelo workflow:

```
docker compose up -d spaceline-db
```

No PowerShell:

```
$env:DATABASE_URL = 'postgresql+psycopg://postgres:postgres@localhost:5432/postgres'
Set-Location apps/server
pytest --cov --cov-report=xml
```

O workflow configura runners e instalação a cada job; os comandos acima reproduzem as verificações, mas não o ambiente isolado de GitHub Actions.

Referências do repositório

- [.github/workflows/quality.yml](#)
- [.github/workflows/autofix.yml](#)
- [.github/workflows/discord-pr.yml](#)
- [apps/mobile/package.json](#)
- [apps/server/package.json](#)
- [apps/server/pyproject.toml](#)
- [apps/server/requirements.txt](#)
- [docker-compose.yml](#)